

Tecnológico Nacional de México / Instituto Tecnológico de Culiacán.



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Ingeniería en Sistemas Computacionales.

Inteligencia Artificial

Zuriel Dathan Mora Félix.

Modulo 3

Sistema experto.

Ontiveros Sánchez Jesús Daniel (22170750).

Ramírez Ruiz Santiago (22170783).

Horario de 9:00 a 10:00 AM.

Culiacán Sinaloa a 6 de noviembre del 2025

Documentación del código.

El presente documento describe el desarrollo y funcionamiento de un Sistema Experto para el diagnóstico de enfermedades respiratorias, diseñado como una herramienta de apoyo clínico basada en Inteligencia Artificial. Este sistema emula el razonamiento de un médico especialista mediante la aplicación de reglas de conocimiento médico, permitiendo analizar los síntomas y signos ingresados por el usuario para ofrecer diagnósticos presuntivos acompañados de su probabilidad, explicación y recomendaciones iniciales.

El sistema está compuesto por tres elementos principales:

- Base de Conocimiento, donde se definen las variables clínicas (síntomas, signos, antecedentes) y las reglas médicas asociadas a enfermedades como asma, neumonía, bronquitis, EPOC, COVID-19, influenza, resfriado común y faringitis.
- Motor de Inferencia, encargado de procesar los datos ingresados y determinar qué reglas se cumplen parcial o totalmente, aplicando factores de certeza y pesos para calcular las probabilidades de cada diagnóstico.
- Interfaz de Usuario, implementada en Streamlit, que permite al usuario ingresar la información del paciente, ejecutar el análisis y visualizar los resultados de forma clara, ordenada y comprensible.

Además, el sistema cuenta con un módulo de pruebas de validación que permite comprobar la coherencia y precisión de las reglas mediante casos clínicos completos, parciales y negativos.

En conjunto, este Sistema Experto constituye una herramienta educativa y técnica que demuestra la aplicación práctica de la inteligencia artificial simbólica en el ámbito médico, promoviendo el razonamiento explicable, la transparencia en los resultados y la posibilidad de extender el conocimiento clínico mediante nuevas reglas o variables.

App.py

```
"""
Interfaz principal del Sistema Experto para diagnóstico de enfermedades respiratorias.
"""

import streamlit as st                                     # Importa Streamlit para construir la
interfaz web
from base_conocimiento import REGLAS, VARIABLES           # Importa reglas y definición de
variables de la BC
from motor_inferencia import (                             # Importa motores de inferencia
    encadenamiento_adelante)

# -----
# Configuración general de la página
# -----
st.set_page_config(                                       # Configura metadatos de la app
    page_title="Sistema Experto Respiratorio",           # Título de la pestaña del navegador
    page_icon="",                                         # Ícono de la página (vacío = ícono por
defecto)
    layout="wide"                                         # Usa layout ancho (full width)
)

# -----
# Utilidades de estado: detectar si el usuario tocó un campo
# -----
def _ensure_state(var: str):
    """Asegura que exista la bandera 'tocado' para la variable dada en session_state."""
    if f"t_{var}" not in st.session_state:                # Si no existe la clave de 'tocado' para
var
        st.session_state[f"t_{var}"] = False             # Inicialízala en False

def _mark_touched(var: str):
    """Marca una variable como 'tocada' (el usuario interactuó)."""
    st.session_state[f"t_{var}"] = True                   # Cambia la bandera a True

def _is_touched(var: str) -> bool:
    """Retorna True si la variable fue 'tocada' por el usuario."""
    return bool(st.session_state.get(f"t_{var}", False))  # Lee la bandera con valor por
defecto False

# -----
# Encabezado principal
# -----
```

```

st.title("Sistema Experto – Diagnóstico de Enfermedades Respiratorias") # Título grande en la
UI
st.caption(
# Subtítulo/ayuda
bajo el título
    "Ingresa datos en el panel **>>> (arriba a la izquierda) para ingresar los datos y
síntomas del paciente y pulsa "
    "***Calcular diagnóstico** para mostrar la probabilidades de cada una de las enfermedades."
)

# -----
# Campo de entrada con detección de interacción
# -----
def input_field(var: str, meta: dict):
    """
    Dibuja el control según tipo, registra 'tocado' con on_change,
    y devuelve (valor, incluir?). Solo incluimos si el usuario tocó el control
    o si el valor es distinto al placeholder (en texto).
    """
    _ensure_state(var) # Asegura que exista la bandera de
'tocado' para var

    tipo = meta["tipo"] # Tipo de dato (booleano, entero,
flotante, texto)
    etiqueta = meta.get("etiqueta", # Etiqueta del control (fallback:
nombre bonito)
                    var.replace("_", " ").capitalize())
    desc = meta.get("descripcion") # Descripción opcional para mostrar
como caption
    key = f"in_{var}" # Clave única del control en Streamlit

    # BOOLEANO: checkbox. Unchecked por defecto NO entra hasta que lo toquen.
    if tipo == "booleano":
        val = st.checkbox( # Renderiza un checkbox
            etiqueta,
            value=False, # Por defecto desmarcado
            key=key, # Clave del widget
            on_change=_mark_touched, # Marca como tocado cuando cambie
            args=(var,) # Pasa el nombre de la variable a
_mark_touched
        )
        if desc: st.caption(desc) # Muestra descripción si existe
        incluir = _is_touched(var) # Solo incluir si el usuario interactuó
        return (val if incluir else None), incluir # Devuelve valor real solo si
incluir=True

```

```

# ENTERO
if tipo == "entero":
    val = st.number_input(
        etiqueta,
        min_value=meta.get("min", 0),
        max_value=meta.get("max", 100),
        value=meta.get("min", 0),
        step=1,
        key=key,
        on_change=_mark_touched,
        args=(var,)
    )
    if desc: st.caption(desc)
    incluir = _is_touched(var)
    return (val if incluir else None), incluir
    incluir

# FLOTANTE
if tipo == "flotante":
    val = st.number_input(
        etiqueta,
        min_value=meta.get("min", 0.0),
        max_value=meta.get("max", 100.0),
        value=meta.get("min", 0.0),
        step=0.1,
        format="%.1f",
        key=key,
        on_change=_mark_touched,
        args=(var,)
    )
    if desc: st.caption(desc)
    incluir = _is_touched(var)
    return (val if incluir else None), incluir
    incluir

# TEXTO: insertamos placeholder inicial que NO cuenta
if tipo == "texto":
    opciones = meta.get("opciones", [])
    placeholder = "- (sin dato) -"
    if opciones:

```

```

    opciones_ui = [placeholder] + opciones    # Inserta placeholder al inicio de la
lista
    idx = st.selectbox(                        # Renderiza un combo/select
        etiqueta,
        options=list(range(len(opciones_ui))), # Índices como opciones internas
        format_func=lambda i: opciones_ui[i], # Muestra el texto correspondiente
        index=0,                               # Selecciona por defecto el placeholder
        key=key,                               # Clave del widget
        on_change=_mark_touched,              # Marca como tocado al cambiar
        args=(var,)                          # Pasa el nombre de la variable
    )
    val = None if idx == 0 else opciones_ui[idx] # Si está en placeholder => None; si
no, valor real
    else:
        # texto libre con placeholder visual
        val = st.text_input(                  # Renderiza un input de texto libre
            etiqueta,
            value="",                         # Vacío por defecto
            key=key,                         # Clave del widget
            on_change=_mark_touched,        # Marca como tocado al cambiar
            args=(var,)                     # Pasa el nombre de la variable
        )
        if val == "":                       # Si quedó vacío, no cuenta
            val = None
        if desc: st.caption(desc)            # Muestra descripción si existe
        incluir = (val is not None)          # Incluye si el valor no es None (i.e.,
no placeholder/ vacío)
        return val, incluir                 # Devuelve valor y bandera incluir

    # Fallback
    if desc: st.caption(desc)               # Si no coincide con tipos conocidos,
solo muestra descripción
    return None, False                     # No incluir por defecto

# -----
# Agrupación visual de variables
# -----
GRUPOS = {                                # Define grupos lógicos para la UI
lateral
    "Datos personales": [
        "edad", "sexo"
    ],
    "Síntomas": [
        "fiebre_c", "tos", "duracion_tos_dias",
        "sibilancias", "disnea", "dolor_pecho", "fatiga",

```

```

        "cefalea", "mialgias", "odinofagia", "anosmia",
        "congestion_nasal", "rinorrea", "exudado_amigdalino",
        "adenopatias_cervicales", "estornudos"
    ],
    "Signos y estudios": [
        "satO2", "crepitantes", "roncus", "sibilos_auscultacion",
        "rx_consolidacion", "pcr_alta", "leucocitosis"
    ],
    "Factores de riesgo": [
        "tabaquismo", "paquetes_por_dia", "anios_fumando",
        "exposicion_contaminantes", "alergias_atopia",
        "infeccion_respiratoria_reciente", "contacto_covid",
        "estacional_invierno"
    ]
}

# -----
# Sección lateral (entrada de datos del paciente)
# -----
with st.sidebar:
    # Inicia el panel lateral
    st.header("Datos del paciente")
    # Encabezado del panel
    hechos = {}
    # Diccionario de hechos (solo campos
incluidos)
    usados = set()
    # Conjunto para rastrear variables
mostradas

    for titulo, llaves in GRUPOS.items():
        # Itera por grupos definidos
        st.subheader(titulo)
        # Subtítulo del grupo
        for k in llaves:
            # Itera por cada variable del grupo
            if k in VARIABLES:
                # Verifica que la var exista en la BC
                val, incluir = input_field(k, VARIABLES[k]) # Dibuja control y obtiene
valor/incluir
                if incluir:
                    # Si debe incluirse
                    hechos[k] = val
                    # Agrega a hechos (evidencias)
                usados.add(k)
                # Marca variable como usada en la UI

    # Variables no agrupadas (si las hubiera)
    restantes = [k for k in VARIABLES.keys() if k not in usados] # Calcula variables que no
están en GRUPOS
    if restantes:
        # Si existen variables restantes
        st.subheader("Otros")
        # Muestra otro subgrupo
        for k in restantes:
            # Itera sobre ellas
            val, incluir = input_field(k, VARIABLES[k]) # Dibuja control
            if incluir:
                # Si se debe incluir
                hechos[k] = val
                # Agrega a hechos

```

```

# -----
# Estructura visual en columnas
# -----
col1, col2 = st.columns([1.3, 1])          # Crea dos columnas (col1 más ancha que
col2)

# -----
# Columna 1: Resultados del diagnóstico
# -----
with col1:                                  # Comienza contenido en la primera
columna
    st.subheader("Presentación del diagnóstico")    # Título de sección

    calcular = st.button("Calcular diagnóstico", type="primary") # Botón principal de acción

    if calcular and hechos:                  # Si se presionó y hay hechos
capturados
        trazas, puntajes, explicaciones, recomendaciones = encadenamiento_adelante( # Llama
motor forward chaining
        hechos,
        REGLAS
        )

        if puntajes:                        # Si hay puntajes (diagnósticos
evaluados)
            orden = sorted(                 # Ordena diagnósticos por probabilidad
descendente
                puntajes.items(),
                key=lambda x: x[1],
                reverse=True
            )
            st.write("**Diagnósticos presuntivos (con probabilidad):**") # Título de lista
            for dx, fc in orden:            # Itera por diagnósticos ordenados
                st.markdown(f"**{dx}** - probabilidad **{fc*100:.1f}%**") # Muestra cada dx
con % formato

            st.divider()                    # Separador visual

            st.subheader("Recomendaciones iniciales") # Subtítulo de recomendaciones
            for dx, _ in orden[:3]:         # Toma top-3 diagnósticos
                if recomendaciones.get(dx):  # Si hay recomendaciones para ese dx
                    st.markdown(f"**{dx}:** " + "; ".join(recomendaciones[dx])) # Lista unida
por ';'

```



```

        st.subheader("Explicación en lenguaje natural") # Subtítulo de explicaciones
        for dx, _ in orden[:3]: # Para top-3 diagnósticos
            if explicaciones.get(dx): # Si hay explicación
                with st.expander(f"¿Por qué se diagnosticó {dx}?"): # Expandir por dx
                    for frase in explicaciones[dx]: # Itera por frases explicativas
                        st.markdown("• " + frase) # Muestra cada razón como viñeta

            else: # Si no se activaron reglas
                st.info("Con los datos proporcionados no se activó ninguna regla. Complete uno o
más campos e intente nuevamente.") # Mensaje informativo

        elif calcular and not hechos: # Si se presionó pero no hay hechos
            st.info("No ingresaste ningún dato. Modifica al menos un campo y vuelve a
calcular.") # Pide ingresar algo

        else: # Si aún no se presiona el botón
            st.caption("Modifica uno o más campos en el panel lateral y pulsa **Calcular
diagnóstico**.") # Instrucción breve

# -----
# Columna 2: Visualización de las reglas de la base de conocimiento
# -----
with col2: # Comienza contenido en la segunda
columna
    st.subheader("Reglas de la Base de Conocimiento") # Título de sección
    st.write(f"Total de reglas: **{len(REGLAS)}**") # Muestra el número total de reglas
cargadas
    for r in REGLAS: # Itera por cada regla
        with st.expander(f"{r['id']} → {r['entonces']} (fc={r.get('fc',1.0)})"): # Expandir
por regla con id, conclusión y fc
            st.json(r) # Muestra la regla completa en formato
JSON (legible)

```

Esta clase construye la interfaz principal del Sistema Experto con Streamlit.

Su función es capturar datos del paciente de forma controlada, ejecutar el motor de inferencia con esos datos y presentar diagnósticos, recomendaciones y explicaciones de manera clara.

Funciones principales:

- Configura la página (título, ícono, diseño ancho).
- Implementa banderas de “campo tocado” (`_ensure_state`, `_mark_touched`, `_is_touched`) para evitar usar valores por defecto.
- Renderiza controles según el tipo de variable (`input_field`): booleano, entero, flotante o texto con placeholder.
- Agrupa variables en el panel lateral (GRUPOS) y construye el diccionario hechos solo con los campos realmente ingresados.
- Ejecuta `encadenamiento_adelante(hechos, REGLAS)` al pulsar “Calcular diagnóstico”.
- Muestra resultados ordenados por probabilidad, recomendaciones para los Top-3 y una explicación expandible por diagnóstico.
- Incluye un visor de reglas: total de reglas y detalle en formato JSON por cada una.
- Maneja estados de uso: guía si no hay datos o si no se activan reglas.

Importancia:

Evita diagnósticos generados por campos no modificados, mejora la trazabilidad del resultado (qué reglas se activaron y por qué) y ofrece una experiencia de usuario clara y auditable. Esta clase conecta la Base de Conocimiento (VARIABLES, REGLAS) con el motor de inferencia, garantizando entradas válidas y salidas interpretables para evaluación clínica o académica.

Motor_inferencia.py

```
"""
Este archivo es mediante el cual funciona el sistema experto.
Aquí se analizan los datos que ingresa el usuario (síntomas, signos, etc.)
y se comparan con las reglas de la base de conocimiento para llegar a
un posible diagnóstico y sus probabilidades.
"""

from typing import Dict, Any, List # Solo sirve para aclarar el tipo de datos (diccionarios, listas,
etc.)

# =====
# FUNCIÓN QUE REVISAS SI UNA CONDICIÓN SE CUMPLE
# =====
# Esta función compara lo que dijo el usuario con lo que espera la regla.

def evaluar(valor_usuario, operador, valor_regla):
    """Compara el valor del usuario con el valor esperado según el operador."""
    if operador == "==": return valor_usuario == valor_regla # Igual
    if operador == "!=": return valor_usuario != valor_regla # Diferente
    if operador == ">=": return valor_usuario >= valor_regla # Mayor o igual
    if operador == "<=": return valor_usuario <= valor_regla # Menor o igual
    if operador == ">": return valor_usuario > valor_regla # Mayor que
    if operador == "<": return valor_usuario < valor_regla # Menor que
    if operador == "in": return valor_usuario in valor_regla # Está dentro de una lista
    if operador == "contiene": return valor_regla in valor_usuario # Contiene una palabra o valor
    return False # Si no se reconoce el operador, se asume que no cumple

# =====
# FUNCIÓN QUE CALCULA QUÉ TANTO SE CUMPLE UNA CONDICIÓN
# =====

def grado_verdad(hechos: Dict[str, Any], condicion: Dict[str, Any]) -> float:
    """
    Calcula si una condición se cumple (1) o no (0).
    También puede usar un "peso" para darle más importancia.
    """
    variable = condicion["variable"] # Nombre de la variable (ej. fiebre_c)
    operador = condicion["operador"] # Operador lógico (==, >, etc.)
    valor = condicion["valor"] # Valor que la regla espera (ej. fiebre_c > 38)
    peso = float(condicion.get("peso", 1.0)) # Importancia de la condición (por defecto vale 1)

    # Si el usuario no puso ese dato, no se puede evaluar
```

```

    if variable not in hechos:
        return 0.0

    # Trata de comparar; si algo sale mal, se considera que no cumple
    try:
        cumple = evaluar(hechos[variable], operador, valor)
    except Exception:
        cumple = False

    # Si cumple → devuelve 1, si no → 0, y lo multiplica por el peso
    return (1.0 if cumple else 0.0) * peso

# =====
# FUNCIONES QUE COMBINAN RESULTADOS DE VARIAS CONDICIONES
# =====

def agregar(valores: List[float], logica: str, pesos: List[float] | None = None) -> float:
    """
    Une los resultados de varias condiciones de una misma regla.
    Si la lógica es 'todas' (AND), se saca un promedio.
    Si la lógica es 'alguna' (OR), se toma el valor más alto.
    """
    if not valores: # Si no hay condiciones, devuelve 0
        return 0.0

    modo = (logica or "todas").strip().lower() # Se limpia el texto

    if modo in {"todas", "and", "y", "all"}:
        # Si todas deben cumplirse → promedio (con pesos si los hay)
        if pesos is None or not pesos:
            pesos = [1.0] * len(valores)
        total_pesos = sum(pesos) or 1.0
        return sum(valores) / total_pesos
    else:
        # Si basta con una (OR), se toma la que más cumple
        return max(valores)

def combinar(existente: float, nuevo: float) -> float:
    """
    Combina dos niveles de certeza para la misma enfermedad.
    Ejemplo: si una regla dice 0.7 y otra 0.5, se combinan sin pasarse de 1.
    """
    return 1.0 - (1.0 - existente) * (1.0 - nuevo)

# =====

```

```

# ENCADENAMIENTO HACIA ADELANTE
# =====
# Aquí se aplican todas las reglas para ver qué diagnósticos se cumplen
# con los datos que dio el usuario.

def encadenamiento_adelante(hechos: Dict[str, Any], reglas: List[Dict[str, Any]]):
    """
    Recorre todas las reglas, calcula cuánto se cumplen y genera:
    - los diagnósticos posibles,
    - las probabilidades,
    - las explicaciones y
    - las recomendaciones.
    """
    trazabilidad, puntajes, explicaciones, recomendaciones = [], {}, {}, {}

    # Se analizan todas las reglas una por una
    for regla in reglas:
        # Se calcula qué tanto se cumple cada condición
        grados = [grado_verdad(hechos, c) for c in regla["si"]]
        pesos = [float(c.get("peso", 1.0)) for c in regla["si"]]
        agregado = agregar(grados, regla.get("logica", "todas"), pesos)
        certeza_regla = agregado * float(regla.get("fc", 1.0)) # Se multiplica por su factor de certeza

        # Si la regla tuvo al menos algo de coincidencia
        if certeza_regla > 0:
            diagnostico = regla["entonces"] # Ejemplo: "Neumonía"
            # Si ya había un valor previo, se combina con el nuevo
            puntajes[diagnostico] = combinar(puntajes.get(diagnostico, 0.0), certeza_regla)

            # Generar una explicación para el usuario
            condiciones_texto = []
            for c in regla["si"]:
                valor = hechos.get(c["variable"])
                try:
                    cumple = evaluar(valor, c["operador"], c["valor"])
                except Exception:
                    cumple = False
                texto = f"{c['variable'].replace('_', ' ')}: {'✅' if cumple else '❌'} (valor: {valor},  

                espera: {c['operador']} {c['valor']}, peso={c.get('peso', 1.0)})"
                condiciones_texto.append(texto)

            # Guarda la explicación completa de la regla
            frase = f"Regla {regla.get('id')}: " + "; ".join(condiciones_texto)
            explicaciones.setdefault(diagnostico, []).append(frase)

```

```

# Si la regla tiene recomendaciones, se guardan
if regla.get("recomendaciones"):
    recomendaciones.setdefault(diagnostico, [])
    for rec in regla["recomendaciones"]:
        if rec not in recomendaciones[diagnostico]:
            recomendaciones[diagnostico].append(rec)

# Guarda un resumen de cómo se aplicó la regla (útil para depurar)
trazabilidad.append({
    "regla_id": regla.get("id"),
    "diagnostico": diagnostico,
    "factor_certeza": round(certeza_regla, 3),
    "cobertura": round(agregado, 3),
})

# Devuelve todos los resultados al programa principal
return trazabilidad, puntajes, explicaciones, recomendaciones

# =====
# ENCADENAMIENTO HACIA ATRÁS
# =====

def encadenamiento_atras(objetivo: str, reglas: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    """
    Busca qué reglas terminan en el diagnóstico indicado.
    Devuelve qué condiciones se deben cumplir para confirmarlo.
    """
    requisitos = []
    for r in reglas:
        if r["entonces"] == objetivo:
            requisitos.append({
                "regla": r.get("id"),
                "logica": r.get("logica", "todas"),
                "fc": r.get("fc", 1.0),
                "condiciones": r["si"],
            })
    return requisitos

```

Esta clase contiene el motor de inferencia del sistema experto.

Su función es comparar los datos ingresados por el usuario con las reglas de la base de conocimiento para determinar posibles diagnósticos y sus probabilidades.

Funciones principales:

- evaluar(): compara el valor del usuario con el esperado según el operador.
- grado_verdad(): calcula si una condición se cumple y aplica su peso.
- agregar(): combina resultados de varias condiciones (modo AND u OR).
- combinar(): une los niveles de certeza de reglas que apuntan al mismo diagnóstico.
- encadenamiento_adelante(): recorre todas las reglas, evalúa coincidencias, genera diagnósticos, explicaciones y recomendaciones.
- encadenamiento_atras(): busca las reglas que concluyen en un diagnóstico dado y muestra las condiciones necesarias.

Importancia:

Es el núcleo lógico del sistema experto. Permite transformar los síntomas del paciente en diagnósticos con certeza cuantificada y justificación, garantizando resultados coherentes y trazables.

Pruebas_validacion.py

```
# pruebas_validacion.py
# -*- coding: utf-8 -*-
"""
Pruebas de validación del Sistema Experto para diagnóstico de enfermedades respiratorias.
- Muestra las probabilidades por diagnóstico y el Top para cada caso.
- Incluye casos COMPLETOS, PARCIALES (aproximados) y NEGATIVOS.
"""

from typing import Dict, Any, List, Tuple
from base_conocimiento import REGLAS
from motor_inferencia import encadenamiento_adelante

# =====
# Helper de impresión
# =====

def mostrar_resultados(nombre: str, hechos: Dict[str, Any]):
    trazas, puntajes, explicaciones, recomendaciones = encadenamiento_adelante(hechos, REGLAS)

    print("\n" + "="*70)
    print(f" Caso: {nombre}")
    print("="*70)

    if not puntajes:
        print("Sin diagnósticos (todas las reglas relevantes quedaron en 0 o no aplican).")
        return

    # Ordenar y mostrar todos los puntajes
    orden = sorted(puntajes.items(), key=lambda kv: kv[1], reverse=True)
    print("\nProbabilidades obtenidas:")
    for dx, sc in orden:
        print(f"   - {dx}: {sc*100:.1f}%")

    # Top 1
    top_dx, top_sc = orden[0]
    print(f"\nTOP: {top_dx} - {top_sc*100:.1f}%")

    # (Opcional) Primera explicación por diagnóstico, si existe
    if explicaciones:
        print("\nExplicaciones generadas:")
        for dx, frases in explicaciones.items():
            if frases:
```



```

        print(f"  {dx}: {frases[0]}")

# (Opcional) Recomendaciones
if recomendaciones:
    print("\nRecomendaciones sugeridas:")
    for dx, recs in recomendaciones.items():
        print(f"  {dx}: {' '.join(recs)}")

print("\n" + "-"*70)

# =====
# Casos COMPLETOS (diagnósticos principales)
# =====

def caso_neumonia_1():
    """Neumonía clínica típica."""
    return dict(
        edad=72, sexo="Femenino", fiebre_c=39.0, tos="Productiva", duracion_tos_dias=5,
        disnea=True, sibilancias=False, dolor_pecho=True, fatiga=True,
        cefalea=False, mialgias=False, odinofagia=False, anosmia=False,
        satO2=90, crepitantes=True, roncus=False, sibilos_auscultacion=False,
        rx_consolidacion=False, pcr_alta=True, leucocitosis=True,
        tabaquismo="Exfumador", paquetes_por_dia=0.5, anios_fumando=10,
        exposicion_contaminantes=False, alergias_atopia=False,
        infeccion_respiratoria_reciente=True, contacto_covid=False,
        estacional_invierno=False
    )

def caso_neumonia_2_rx():
    """Neumonía por imagen + labs (evitando disparar la clínica completa)."""
    return dict(
        edad=65, sexo="Masculino",
        fiebre_c=38.2, tos="Seca", duracion_tos_dias=2,
        disnea=False, sibilancias=False, dolor_pecho=False, fatiga=True,
        cefalea=False, mialgias=False, odinofagia=False, anosmia=False,
        satO2=95, crepitantes=False, roncus=False, sibilos_auscultacion=False,
        rx_consolidacion=True, pcr_alta=True, leucocitosis=True,
        tabaquismo="No", paquetes_por_dia=0.0, anios_fumando=0,
        exposicion_contaminantes=False, alergias_atopia=False,
        infeccion_respiratoria_reciente=False, contacto_covid=False,
        estacional_invierno=False
    )

def caso_asma_1():
    """Asma (tos seca + alergia + sibilancias)."""

```

```

    return dict(
        edad=22, sexo="Masculino", fiebre_c=36.9, tos="Seca", duracion_tos_dias=10,
        disnea=False, sibilancias=True, dolor_pecho=False, fatiga=False,
        cefalea=False, mialgias=False, odinofagia=False, anosmia=False,
        satO2=97, crepitantes=False, roncus=False, sibilos_auscultacion=False,
        rx_consolidacion=False, pcr_alta=False, leucocitosis=False,
        tabaquismo="No", paquetes_por_dia=0.0, anios_fumando=0,
        exposicion_contaminantes=False, alergias_atopia=True,
        infeccion_respiratoria_reciente=False, contacto_covid=False,
        estacional_invierno=False
    )

def caso_asma_2():
    """Asma por esfuerzo (disnea + sibilancias + sibilos en auscultación)."""
    return dict(
        edad=28, sexo="Femenino",
        fiebre_c=36.7, tos="Seca", duracion_tos_dias=3,
        disnea=True, sibilancias=True, dolor_pecho=False, fatiga=False,
        cefalea=False, mialgias=False, odinofagia=False, anosmia=False,
        satO2=98, crepitantes=False, roncus=False, sibilos_auscultacion=True,
        rx_consolidacion=False, pcr_alta=False, leucocitosis=False,
        tabaquismo="No", paquetes_por_dia=0.0, anios_fumando=0,
        exposicion_contaminantes=True, alergias_atopia=False,
        infeccion_respiratoria_reciente=False, contacto_covid=False,
        estacional_invierno=False
    )

def caso_epoc_1():
    """EPOC típico, fumador intenso con disnea + sibilancias."""
    return dict(
        edad=65, sexo="Masculino",
        fiebre_c=36.8, tos="Seca", duracion_tos_dias=60,
        disnea=True, sibilancias=True, dolor_pecho=False, fatiga=True,
        cefalea=False, mialgias=False, odinofagia=False, anosmia=False,
        satO2=93, crepitantes=False, roncus=True, sibilos_auscultacion=True,
        rx_consolidacion=False, pcr_alta=False, leucocitosis=False,
        tabaquismo="Actual", paquetes_por_dia=1.2, anios_fumando=30,
        exposicion_contaminantes=True, alergias_atopia=False,
        infeccion_respiratoria_reciente=False, contacto_covid=False,
        estacional_invierno=False
    )

def caso_epoc_2():
    """EPOC moderado (evita activar ASMA_2 apagando sibilos en auscultación)."""
    return dict(

```

```

edad=58, sexo="Masculino", fiebre_c=36.9, tos="Productiva", duracion_tos_dias=40,
disnea=True, sibilancias=True, dolor_pecho=False, fatiga=True,
cefalea=False, mialgias=False, odinofagia=False, anosmia=False,
satO2=94, crepitantes=False, roncus=True,
sibilos_auscultacion=False, # clave
rx_consolidacion=False, pcr_alta=False, leucocitosis=False,
tabaquismo="Exfumador", paquetes_por_dia=0.6, anios_fumando=20,
exposicion_contaminantes=True, alergias_atopia=False,
infeccion_respiratoria_reciente=False, contacto_covid=False,
estacional_invierno=False
)

def caso_covid_1():
    """COVID-19 (fiebre + tos + fatiga + contacto)."""
    return dict(
        edad=35, sexo="Femenino",
        fiebre_c=38.0, tos="Seca", duracion_tos_dias=3,
        disnea=False, sibilancias=False, dolor_pecho=False, fatiga=True,
        cefalea=True, mialgias=True, odinofagia=False, anosmia=False,
        satO2=97, crepitantes=False, roncus=False, sibilos_auscultacion=False,
        rx_consolidacion=False, pcr_alta=False, leucocitosis=False,
        contacto_covid=True,
        tabaquismo="No", paquetes_por_dia=0.0, anios_fumando=0,
        exposicion_contaminantes=False, alergias_atopia=False,
        infeccion_respiratoria_reciente=True,
        estacional_invierno=False
    )

def caso_covid_2_anosmia():
    """COVID-19 olfativo (anosmia + fiebre leve + odinofagia)."""
    return dict(
        edad=26, sexo="Masculino",
        fiebre_c=37.6, tos="No", duracion_tos_dias=0,
        disnea=False, sibilancias=False, dolor_pecho=False, fatiga=False,
        cefalea=False, mialgias=False, odinofagia=True, anosmia=True,
        satO2=98, crepitantes=False, roncus=False, sibilos_auscultacion=False,
        rx_consolidacion=False, pcr_alta=False, leucocitosis=False,
        contacto_covid=False,
        tabaquismo="No", paquetes_por_dia=0.0, anios_fumando=0,
        exposicion_contaminantes=False, alergias_atopia=False,
        infeccion_respiratoria_reciente=False,
        estacional_invierno=False
    )

def caso_influenza_1():

```

```

"""Influenza (fiebre alta + mialgias + cefalea + invierno)."""
return dict(
    edad=30, sexo="Femenino",
    fiebre_c=38.5, tos="Seca", duracion_tos_dias=2,
    disnea=False, sibilancias=False, dolor_pecho=False, fatiga=True,
    cefalea=True, mialgias=True, odinofagia=False, anosmia=False,
    satO2=98, crepitantes=False, ronus=False, sibilos_auscultacion=False,
    rx_consolidacion=False, pcr_alta=False, leucocitosis=False,
    contacto_covid=False,
    tabaquismo="No", paquetes_por_dia=0.0, anios_fumando=0,
    exposicion_contaminantes=False, alergias_atopia=False,
    infeccion_respiratoria_reciente=False,
    estacional_invierno=True
)

def caso_resfriado_1():
    """Resfriado nasal típico (rinorrea clara + congestión + fiebre baja)."""
    return dict(
        edad=18, sexo="Masculino",
        fiebre_c=37.2, tos="Seca", duracion_tos_dias=1,
        disnea=False, sibilancias=False, dolor_pecho=False, fatiga=False,
        cefalea=False, mialgias=False, odinofagia=True, anosmia=False,
        satO2=99, crepitantes=False, ronus=False, sibilos_auscultacion=False,
        rx_consolidacion=False, pcr_alta=False, leucocitosis=False,
        rinorrea="Clara", congestion_nasal=True, estornudos=True,
        contacto_covid=False,
        tabaquismo="No", paquetes_por_dia=0.0, anios_fumando=0,
        exposicion_contaminantes=False, alergias_atopia=True,
        infeccion_respiratoria_reciente=False,
        estacional_invierno=False
    )

def caso_resfriado_2():
    """Resfriado leve con tos ausente (validando otra regla)."""
    return dict(
        edad=22, sexo="Femenino",
        fiebre_c=37.0, tos="No", duracion_tos_dias=0,
        disnea=False, sibilancias=False, dolor_pecho=False, fatiga=False,
        cefalea=False, mialgias=False, odinofagia=False, anosmia=False,
        satO2=99, crepitantes=False, ronus=False, sibilos_auscultacion=False,
        rx_consolidacion=False, pcr_alta=False, leucocitosis=False,
        rinorrea="Clara", congestion_nasal=True, estornudos=True,
        contacto_covid=False,
        tabaquismo="No", paquetes_por_dia=0.0, anios_fumando=0,
        exposicion_contaminantes=False, alergias_atopia=True,

```

```

        infeccion_respiratoria_reciente=False,
        estacional_invierno=False
    )

def caso_resfriado_3():
    """Resfriado nasal claro (tercera variante de reglas)."""
    return dict(
        edad=25, sexo="Masculino",
        fiebre_c=37.4, tos="Seca", duracion_tos_dias=2,
        disnea=False, sibilancias=False, dolor_pecho=False, fatiga=False,
        cefalea=False, mialgias=False, odinofagia=False, anosmia=False,
        satO2=99, crepitantes=False, roncus=False, sibilos_auscultacion=False,
        rx_consolidacion=False, pcr_alta=False, leucocitosis=False,
        rinorrea="Clara", congestion_nasal=True, estornudos=True,
        contacto_covid=False,
        tabaquismo="No", paquetes_por_dia=0.0, anios_fumando=0,
        exposicion_contaminantes=False, alergias_atopia=True,
        infeccion_respiratoria_reciente=False,
        estacional_invierno=False
    )

def caso_faringitis_viral():
    """Faringitis viral (sin tos para no activar bronquitis)."""
    return dict(
        edad=19, sexo="Femenino",
        fiebre_c=37.6, tos="No", duracion_tos_dias=2,
        disnea=False, sibilancias=False, dolor_pecho=False, fatiga=False,
        cefalea=False, mialgias=False, odinofagia=True, anosmia=False,
        satO2=99, crepitantes=False, roncus=False, sibilos_auscultacion=False,
        rx_consolidacion=False, pcr_alta=False, leucocitosis=False,
        rinorrea="Purulenta", congestion_nasal=False,
        contacto_covid=False,
        tabaquismo="No", paquetes_por_dia=0.0, anios_fumando=0,
        exposicion_contaminantes=False, alergias_atopia=False,
        infeccion_respiratoria_reciente=False,
        estacional_invierno=False
    )

def caso_faringitis_bacteriana():
    """Faringitis estreptocócica (Centor alto)."""
    return dict(
        edad=21, sexo="Masculino",
        fiebre_c=38.2, tos="No", duracion_tos_dias=1,
        disnea=False, sibilancias=False, dolor_pecho=False, fatiga=False,
        cefalea=False, mialgias=False, odinofagia=True, anosmia=False,

```

```

        satO2=99, crepitantes=False, roncus=False, sibilos_auscultacion=False,
        rx_consolidacion=False, pcr_alta=False, leucocitosis=False,
        exudado_amigdalino=True, adenopatias_cervicales=True,
        rinorrea="No", congestión_nasal=False,
        contacto_covid=False,
        tabaquismo="No", paquetes_por_dia=0.0, años_fumando=0,
        exposición_contaminantes=False, alergias_atopia=False,
        infección_respiratoria_reciente=False,
        estacional_invierno=False
    )

def caso_bronquitis_1():
    """Bronquitis aguda pos-IR, subaguda, sin consolidación."""
    return dict(
        edad=35, sexo="Masculino", fiebre_c=37.5,
        tos="Productiva", duración_tos_dias=10,
        infección_respiratoria_reciente=True,
        rx_consolidacion=False,
        congestión_nasal=False, rinorrea="Purulenta",
        disnea=False, crepitantes=False
    )

def caso_bronquitis_2():
    """Bronquitis aguda leve: tos + fiebre baja + sin crepitantes."""
    return dict(
        edad=29, sexo="Femenino",
        fiebre_c=37.8, tos="Seca", duración_tos_dias=5,
        crepitantes=False, rx_consolidacion=False,
        congestión_nasal=False
    )

# =====
# Casos PARCIALES (no se cumplen todas las condiciones; ahora dan aproximado)
# =====

def caso_neumonia_parcial():
    """NEUMONÍA_1 PARCIAL: falta 'crepitantes=True' → esa regla aporta parcial."""
    return dict(
        edad=70, sexo="Femenino",
        fiebre_c=39.0, tos="Productiva", duración_tos_dias=5,
        disnea=True, crepitantes=False, # falta crepitantes
        sibilancias=False, dolor_pecho=True, fatiga=True,
        rx_consolidacion=False, leucocitosis=False, pcr_alta=False,
        roncus=False, sibilos_auscultacion=False,
        satO2=92, contacto_covid=False, infección_respiratoria_reciente=False
    )

```

```

    )

def caso_asma_parcial():
    """ASMA_1 PARCIAL: falta 'alergias_atopia=True'."""
    return dict(
        edad=25, sexo="Masculino",
        tos="Seca", sibilancias=True, alergias_atopia=False, # falta alergia
        disnea=False, sibilos_auscultacion=False, rx_consolidacion=False
    )

def caso_bronquitis_1_parcial_con_bronquitis_2_activa():
    """BRONQUITIS_1 PARCIAL, pero BRONQUITIS_2 sí aporta."""
    return dict(
        edad=33, sexo="Masculino",
        tos="Productiva", duracion_tos_dias=10,
        infeccion_respiratoria_reciente=False, # falta para BRONQUITIS_1
        rx_consolidacion=False, crepitantes=False,
        fiebre_c=37.6, congestion_nasal=False # activa BRONQUITIS_2
    )

def caso_covid_1_parcial_con_covid_2_activa():
    """COVID_1 PARCIAL (sin contacto), pero COVID_2 activa (anosmia + odinofagia)."""
    return dict(
        edad=29, sexo="Femenino",
        fiebre_c=37.6, tos="No", fatiga=False,
        contacto_covid=False, # falta para COVID_1
        anosmia=True, odinofagia=True, # activa COVID_2
        rx_consolidacion=False, leucocitosis=False, pcr_alta=False
    )

def caso_faringitis_bacteriana_parcial_con_viral_activa():
    """Faringitis bacteriana parcial (tos 'Seca'), viral activa."""
    return dict(
        edad=20, sexo="Femenino",
        fiebre_c=37.5, odinofagia=True, tos="Seca", # rompe la bacteriana
        congestion_nasal=False, rinorrea="Purulenta", # activa la viral
        rx_consolidacion=False, leucocitosis=False
    )

# =====
# PRUEBAS NEGATIVAS – No deben arrojar diagnósticos
# =====

def caso_negativo_vacio():
    """Sin hechos: agregado=0 → sin diagnósticos."""

```

```

    return dict()

def caso_negativo_demografia_sola():
    """Solo demografía: ninguna regla se activa por edad/sexo únicamente."""
    return dict(
        edad=35,
        sexo="Masculino"
    )

def caso_negativo_categorias_fuera_de_opciones():
    """Categóricos fuera del catálogo; omitimos umbrales y RX."""
    return dict(
        tos="Indefinida",
        rinorrea="Ninguna",
        tabaquismo="Desconocido",
        congestion_nasal=None,
        disnea=None, sibilancias=None
    )

def caso_negativo_minimos_inofensivos():
    """Valores que no activan reglas (evitamos fiebre_c, RX, labs, nasales)."""
    return dict(
        tos="No",
        disnea=False,
        sibilancias=False,
        crepitantes=False,
        sibilos_auscultacion=False,
        alergias_atopia=False,
        contacto_covid=False,
        estacional_invierno=False
    )

def caso_negativo_labs_sueltos_inconclusos():
    """Labs sueltos no suficientes para activar neumonía RX."""
    return dict(
        pcr_alta=True,
        leucocitosis=False,
        tos="No",
        disnea=False,
        sibilancias=False,
        congestion_nasal=False
    )

# =====
# Listas de pruebas a correr

```



```

# =====

PRUEBAS_COMPLETAS = [
    ("Neumonía (clínica)", caso_neumonia_1),
    ("Neumonía (RX+labs)", caso_neumonia_2_rx),
    ("Asma_1", caso_asma_1),
    ("Asma_2", caso_asma_2),
    ("EPOC_1", caso_epoc_1),
    ("EPOC_2", caso_epoc_2),
    ("COVID_1", caso_covid_1),
    ("COVID_2_ANOSMIA", caso_covid_2_anosmia),
    ("Influenza_1", caso_influenza_1),
    ("Resfriado_1", caso_resfriado_1),
    ("Resfriado_2", caso_resfriado_2),
    ("Resfriado_3", caso_resfriado_3),
    ("Faringitis (viral)", caso_faringitis_viral),
    ("Faringitis (bacteriana)", caso_faringitis_bacteriana),
    ("Bronquitis_1", caso_bronquitis_1),
    ("Bronquitis_2", caso_bronquitis_2),
]

PRUEBAS_PARCIALES = [
    ("NEUMONÍA_1 parcial (sin crepitantes)", caso_neumonia_parcial),
    ("ASMA parcial (sin alergia ni esfuerzo)", caso_asma_parcial),
    ("BRONQUITIS_1 parcial, pero BRONQUITIS_2 activa", caso_bronquitis_1_parcial_con_bronquitis_2_activa),
    ("COVID_1 parcial, pero COVID_2 activa", caso_covid_1_parcial_con_covid_2_activa),
    ("Faringitis bacteriana parcial, viral activa", caso_faringitis_bacteriana_parcial_con_viral_activa),
]

PRUEBAS_NEGATIVAS = [
    ("NEGATIVO: vacío", caso_negativo_vacio),
    ("NEGATIVO: demografía sola", caso_negativo_demografia_sola),
    ("NEGATIVO: categorías fuera de opciones", caso_negativo_categorias_fuera_de_opciones),
    ("NEGATIVO: mínimos inofensivos", caso_negativo_minimos_inofensivos),
    ("NEGATIVO: labs sueltos inconclusos", caso_negativo_labs_sueltos_inconclusos),
]

# =====
# Ejecución
# =====

if __name__ == "__main__":
    print("\n" + "#"*70)
    print("# PRUEBAS COMPLETAS")
    print("#"*70)

```

```

for nombre, fabrica in PRUEBAS_COMPLETAS:
    mostrar_resultados(nombre, fabrica())

print("\n" + "#" * 70)
print("# PRUEBAS PARCIALES (no se cumplen todas las condiciones; se muestra aproximado)")
print("#" * 70)
for nombre, fabrica in PRUEBAS_PARCIALES:
    mostrar_resultados(nombre, fabrica())

print("\n" + "#" * 70)
print("# PRUEBAS NEGATIVAS (no deben arrojar diagnósticos)")
print("#" * 70)
for nombre, fabrica in PRUEBAS_NEGATIVAS:
    mostrar_resultados(nombre, fabrica())

```

Este módulo ejecuta pruebas de validación del sistema experto en consola.

Su función es construir casos de prueba (completos, parciales y negativos), invocar el motor de inferencia y mostrar probabilidades, TOP, primeras explicaciones y recomendaciones para verificar el comportamiento.

Funciones principales:

- Define `mostrar_resultados(nombre, hechos)` para llamar `encadenamiento_adelante`, ordenar puntajes, imprimir TOP, explicaciones y recomendaciones.
- Provee casos completos por diagnóstico (neumonía, asma, EPOC, COVID-19, influenza, resfriado, faringitis, bronquitis).
- Provee casos parciales donde faltan condiciones clave para validar aportes incompletos y combinaciones de reglas.
- Provee casos negativos que no deben activar diagnósticos (vacío, solo demografía, categorías inválidas, mínimos inofensivos, labs inconclusos).
- Organiza tres listas (`PRUEBAS_COMPLETAS`, `PRUEBAS_PARCIALES`, `PRUEBAS_NEGATIVAS`) y las ejecuta en `__main__`.

Importancia:

Permite validar rápida y sistemáticamente la base de conocimiento y el motor (AND/OR, pesos, fc, combinación de evidencias). Asegura trazabilidad en consola y detecta regresiones al modificar reglas o variables.